



AMR MOHAMED ZHRAN
Faculty of Engineering
Kafr El-Sheikh University
FPGA – NTI
Project Report For
UART_TX

Introduction

- UART (Universal Asynchronous Receiver-Transmitter) is one of the most widely used serial communication protocols for point-to-point data transmission.
- It allows two digital systems (such as FPGAs, Microcontrollers, and PCs) to communicate reliably without sharing a common clock signal.
- It transmits data one bit at a time, using configurable parameters such as baud rate, parity, stop bits, parity type, and data word length.
- Responsible for serializing and sending data frames.

Project focus

- Designing and implementing a synthesizable UART Transmitter (UART_TX) module using Verilog HDL.

Main purpose

Converting 8-bit parallel input data into a standardized serial data stream with precise bit-timing.

Stage 1: Serializer Block Design

- Core Function: Converts parallel input data into a serial bitstream for transmission.
- Parametrized Data Width: Configured as 8-bit by default, but scalable via Verilog parameters.
- Data Valid Handshake: Uses a `data_valid` enable signal to latch input data into internal registers when ready.
- Shift Register Logic: Holds the parallel data securely upon receiving the valid enable.
- Bit Shifting Mechanism: Shifts out data bit-by-bit using concatenation/shifting logic triggered by the `shift_enable` control signal.

Stage 2: Parity Generation Block

- Calculates and appends an error-detection bit to ensure data integrity during transmission.
- Configurable Parity Modes:
 - Even Parity: Sets the parity bit to make the total number of 1s even.(using xor to the input data)
 - Odd Parity: Sets the parity bit to make the total number of 1s odd.(using xnor to the input data)
- Control & Enable: Generates the parity bit based on a parity enable configuration parameter and a parity type selection flag.
- Internal Storage / Latching: Computes and holds the parity bit in an internal register when data_valid is high, keeping it stable throughout the entire transmission frame.

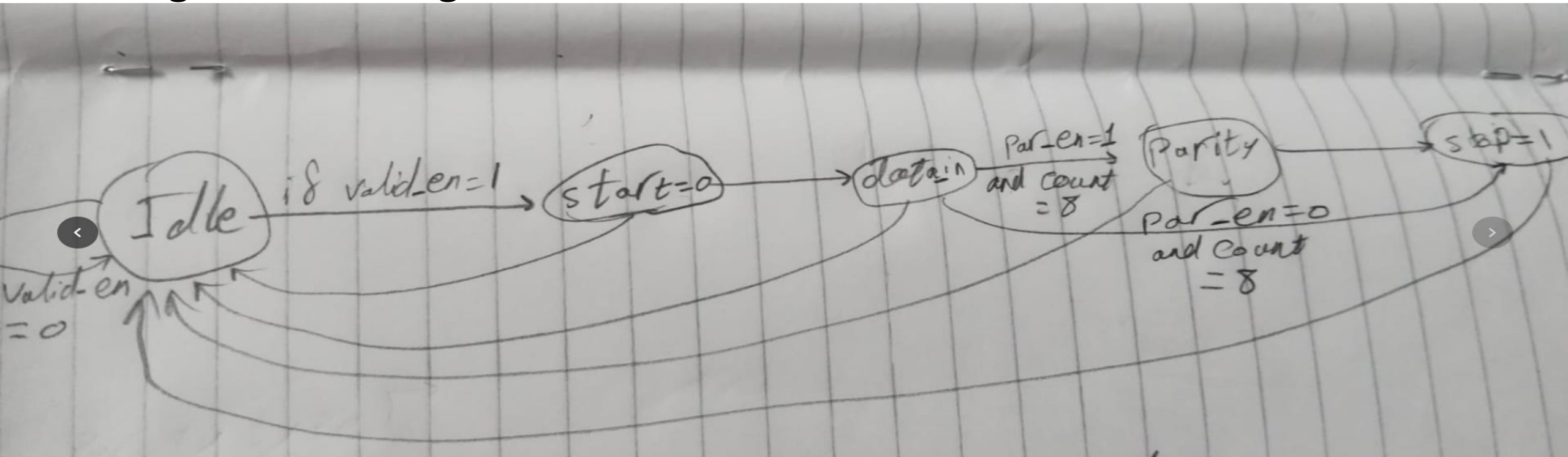
Stage 3: Multiplexer (MUX) Block

- Core Function: Selects and routes the appropriate serial bit to the main tx output line based on the FSM state.
- Input Lines to the MUX:
 - 1'b1 (Idle state line / Stop bit).
 - 1'b0 (Start bit).
 - ser_data (Serialized data bit coming from the Shift Register).
 - parity_bit (Calculated parity bit).
- Control Signal (mux_sel): Driven directly by the FSM to switch between frame segments sequentially.
- Default Output: Kept driven HIGH (Logic 1) during the IDLE state to indicate no transmission is active.

Stage 4: FSM Controller (Finite State Machine)

- Core Function: Acts as the central brain of the transmitter, controlling data flow and timing across all states.
- State Definitions:
 - IDLE: Line stays HIGH (1). Waits for data_valid trigger.
 - START: Asserts the Start Bit (0) onto the line to alert the receiver.
 - DATA: Enables the serializer (shift_enable) and shifts out all 8 data bits sequentially.
 - PARITY: Sends the calculated parity bit (if parity is enabled).
 - STOP: Drives the Stop Bit (1) to close the frame and asserts busy = 0 / tx_out = 1.

- State Transition Controls: Moves between states based on internal counters (baud tick and bit counter completion).
- MUX & Datapath Coordination: Drives the mux_sel lines and status flags at each stage of the frame.



Top Module: System Integration & Architecture

- Core Role: Integrates the sub-modules (Serializer, Parity Calculator, MUX, and FSM Controller) into a unified UART Transmitter core.
- Top-Level Interface:
 - System Inputs: clk, rst_n (Clock & Asynchronous Active-low Reset).
 - User Control Inputs: data_in[7:0], data_valid, parity_enable, parity_type.
 - Outputs: tx_out (Serial output line), busy (Handshake / Transmission status flag).

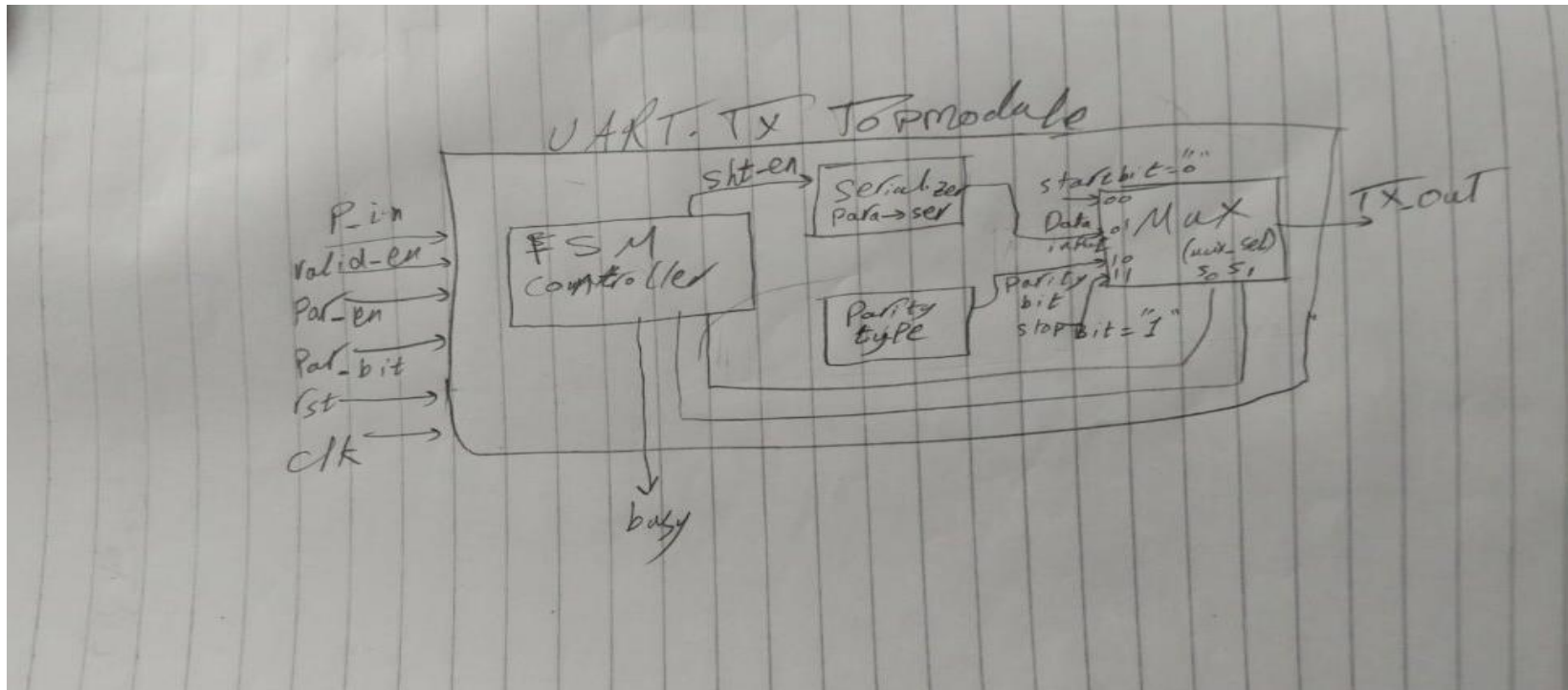
- Internal Interconnects:

Routes control flags (shift_enable, mux_sel) from the FSM to the datapath blocks.

Feeds serialized data (ser_data) and calculated parity (parity_bit) directly to the MUX for serial transmission.

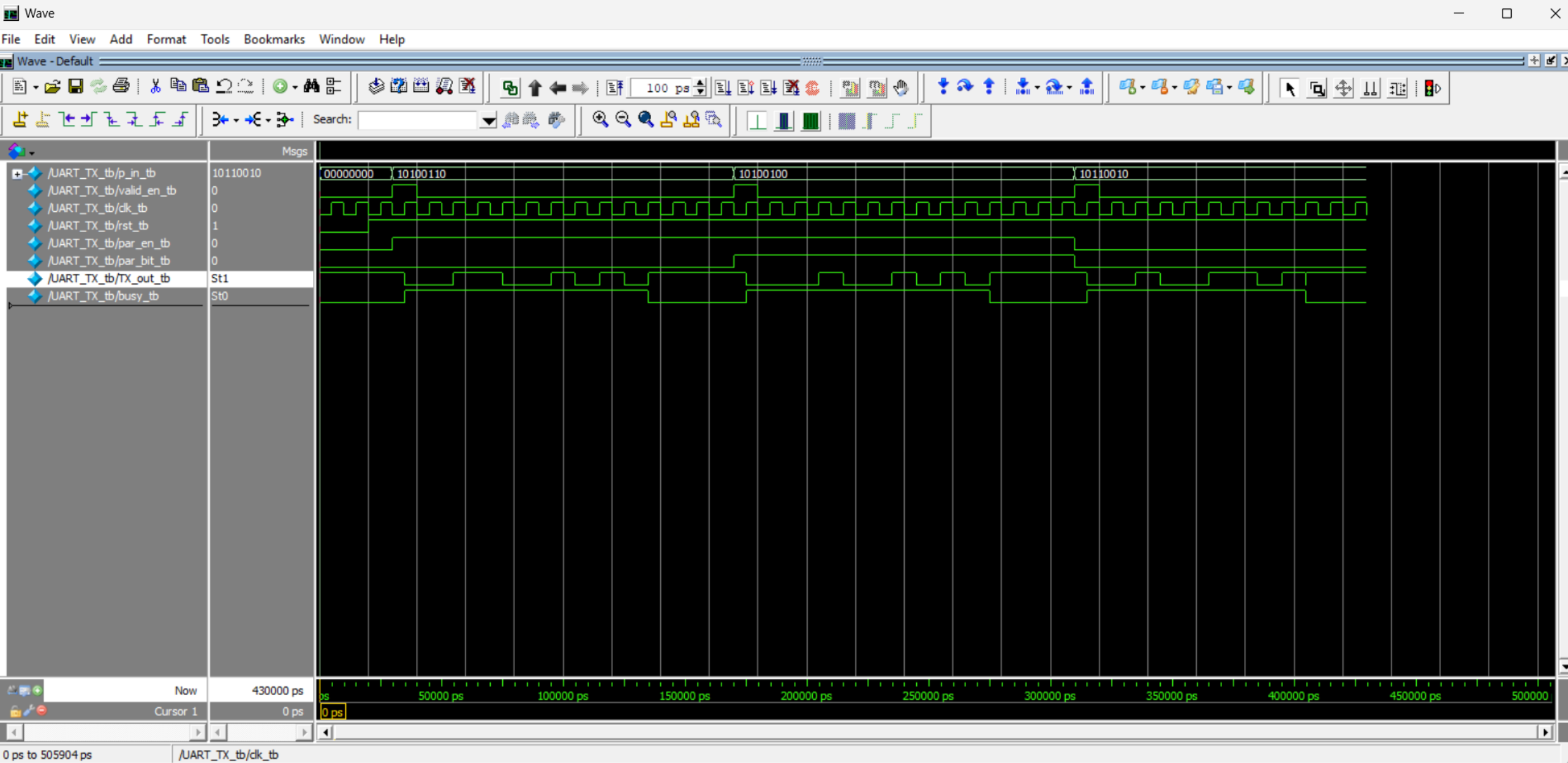
- Hierarchical Design: Clean modular separation between the Control Path (FSM) and Datapath (Registers & MUX).

Block diagram of top module



Verification & Simulation Results

- Testbench Architecture: Self-checking testbench generating clock, reset, and stimulus frames while monitoring outputs.
- Applied Test Scenarios:
 - Scenario 1: Standard 8-bit frame transmission with Even Parity.
 - Scenario 2: Frame transmission with Odd Parity verification.
 - Scenario 3: Back-to-back frames and verification of the busy / tx_out handshake flags.
- Verification Verdict: All test vectors matched the expected serialframe format with zero timing violations.



```
# TIME=0 | p_in=00000000 , valid_en=0 , par_en=0 , par_bit=0 | TX_out=1 , busy=0
#
# ____ scenario 1 ____
# TIME=30000 | p_in=10100110 , valid_en=1 , par_en=1 , par_bit=0 | TX_out=1 , busy=0
# TIME=35000 | p_in=10100110 , valid_en=1 , par_en=1 , par_bit=0 | TX_out=0 , busy=1
# TIME=40000 | p_in=10100110 , valid_en=0 , par_en=1 , par_bit=0 | TX_out=0 , busy=1
# TIME=55000 | p_in=10100110 , valid_en=0 , par_en=1 , par_bit=0 | TX_out=1 , busy=1
# TIME=75000 | p_in=10100110 , valid_en=0 , par_en=1 , par_bit=0 | TX_out=0 , busy=1
# TIME=95000 | p_in=10100110 , valid_en=0 , par_en=1 , par_bit=0 | TX_out=1 , busy=1
# TIME=105000 | p_in=10100110 , valid_en=0 , par_en=1 , par_bit=0 | TX_out=0 , busy=1
# TIME=115000 | p_in=10100110 , valid_en=0 , par_en=1 , par_bit=0 | TX_out=1 , busy=1
# TIME=125000 | p_in=10100110 , valid_en=0 , par_en=1 , par_bit=0 | TX_out=0 , busy=1
# TIME=135000 | p_in=10100110 , valid_en=0 , par_en=1 , par_bit=0 | TX_out=1 , busy=0
#
# ____ scenario 2 ____
# TIME=170000 | p_in=10100100 , valid_en=1 , par_en=1 , par_bit=1 | TX_out=1 , busy=0
# TIME=175000 | p_in=10100100 , valid_en=1 , par_en=1 , par_bit=1 | TX_out=0 , busy=1
# TIME=180000 | p_in=10100100 , valid_en=0 , par_en=1 , par_bit=1 | TX_out=0 , busy=1
# TIME=205000 | p_in=10100100 , valid_en=0 , par_en=1 , par_bit=1 | TX_out=1 , busy=1
# TIME=215000 | p_in=10100100 , valid_en=0 , par_en=1 , par_bit=1 | TX_out=0 , busy=1
# TIME=235000 | p_in=10100100 , valid_en=0 , par_en=1 , par_bit=1 | TX_out=1 , busy=1
# TIME=245000 | p_in=10100100 , valid_en=0 , par_en=1 , par_bit=1 | TX_out=0 , busy=1
# TIME=255000 | p_in=10100100 , valid_en=0 , par_en=1 , par_bit=1 | TX_out=1 , busy=1
# TIME=265000 | p_in=10100100 , valid_en=0 , par_en=1 , par_bit=1 | TX_out=0 , busy=1
# TIME=275000 | p_in=10100100 , valid_en=0 , par_en=1 , par_bit=1 | TX_out=1 , busy=0
#
# ____ scenario 3 ____
# TIME=310000 | p_in=10110010 , valid_en=1 , par_en=0 , par_bit=0 | TX_out=1 , busy=0
# TIME=315000 | p_in=10110010 , valid_en=1 , par_en=0 , par_bit=0 | TX_out=0 , busy=1
# TIME=320000 | p_in=10110010 , valid_en=0 , par_en=0 , par_bit=0 | TX_out=0 , busy=1
# TIME=335000 | p_in=10110010 , valid_en=0 , par_en=0 , par_bit=0 | TX_out=1 , busy=1
# TIME=345000 | p_in=10110010 , valid_en=0 , par_en=0 , par_bit=0 | TX_out=0 , busy=1
# TIME=365000 | p_in=10110010 , valid_en=0 , par_en=0 , par_bit=0 | TX_out=1 , busy=1
# TIME=385000 | p_in=10110010 , valid_en=0 , par_en=0 , par_bit=0 | TX_out=0 , busy=1
# TIME=395000 | p_in=10110010 , valid_en=0 , par_en=0 , par_bit=0 | TX_out=1 , busy=1
# TIME=405000 | p_in=10110010 , valid_en=0 , par_en=0 , par_bit=0 | TX_out=1 , busy=0
# ** Note: $finish : D:/modelsim projects/TX_UART/UART_TX_tb.v(54)
```